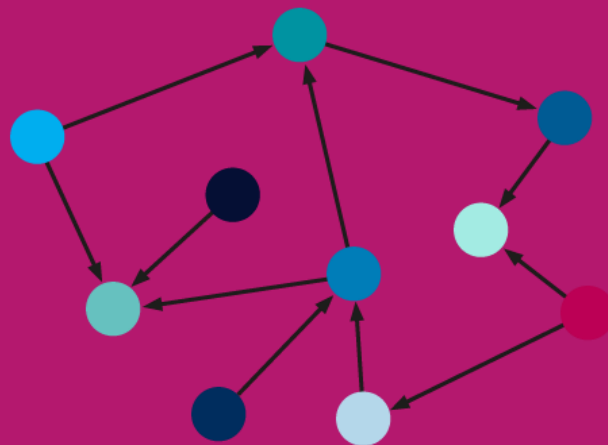




MongoDB

зберігання, резервні копії, безпека та моделювання

Жишкович Андрій Володимирович

Зберігання файлів в MongoDB

Зберігання файлів (зображень, аудіо, відео тощо)

MongoDB може зберігати різні типи файлів.

Існує кілька способів зробити це:

- Зберігання як двійкових даних (Binary Data)
 - GridFS для зберігання великих файлів
- або
- Зберігання URL



Зберігання даних: Двійкові дані (Binary Data)

Ви можете зберігати файли безпосередньо в колекції MongoDB як двійкові дані (Binary Data) за допомогою типу даних **BinData**.

Цей тип даних дозволяє зберігати будь-які дані у двійковому форматі, такі як зображення, аудіо, відео, файли та інші типи даних, які не можуть бути представлені у вигляді тексту.

Це підходить для невеликих файлів до 16Мб.

BinData(<sub_type>,<buffer>)

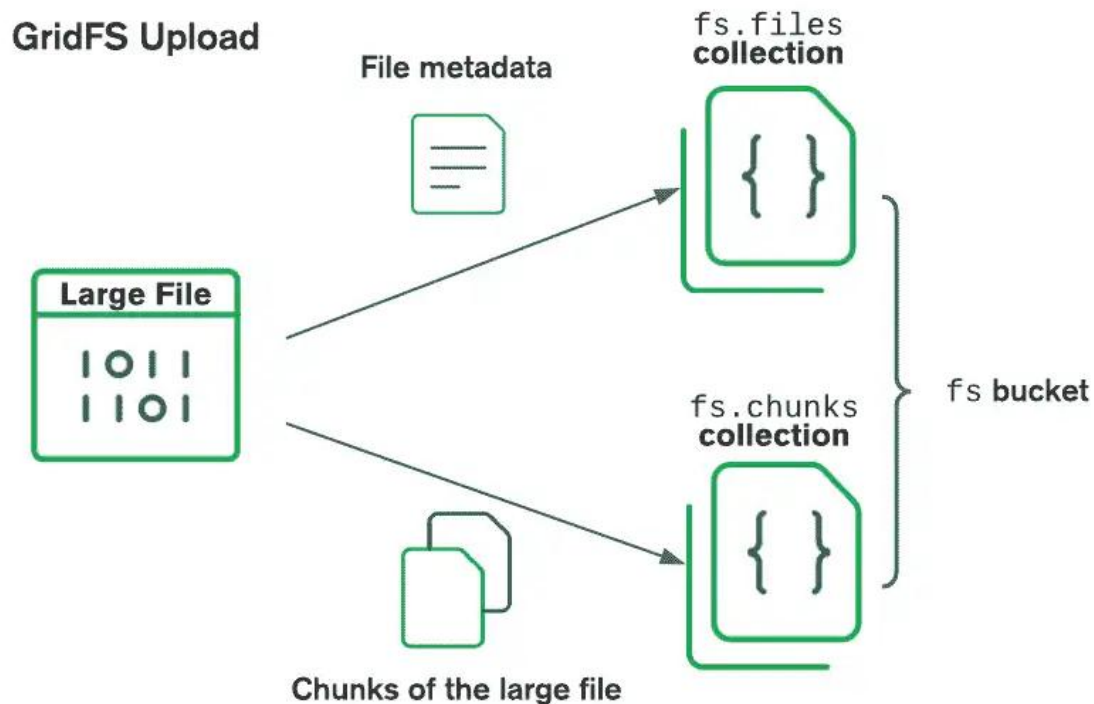
sub_type – Ціле число, яке вказує на тип двійкових даних.

buffer – Об'єкт буфера, що містить двійкові дані. Має бути значення рядка в кодуванні base64.

```
> db.BinCol.find().pretty()
{
  "_id" : ObjectId("55c23c3ef92942ee1fcd5a74"),
  "Comment" : "<bindata> is the base64 representation of a binary string.",
  "Value" : BinData(0,"5465sdf6465sdf65sf56sdf656565sdf")
}
```

Номер	Опис
0	Загальний двійковий підтип (Generic binary subtype)
1	Дані функції (Function data)
2	Двійкові дані (старий формат) (Binary (old))
3	UUID (старий формат) (UUID (old))
4	UUID (стандартний формат) (UUID)
5	MD5
6	Зашифроване значення BSON (Encrypted BSON value)
7	Стиснені дані часових рядів (Compressed time series data)
8	Чутливі дані, такі як ключ або секрет. MongoDB не записує буквальні значення для двійкових даних з підтипом 8. Замість цього MongoDB записує значення-заповнювач ###. (Sensitive data, such as a key or secret. MongoDB does not log literal values for binary data with subtype 8. Instead, MongoDB logs a placeholder value of ##).
9	Вектор (Vector)
128	Користувачькі дані (Custom data)

Зберігання даних: GridFS



Якщо вам потрібно зберігати файли, які перевищують 16МБ, рекомендується використовувати GridFS.

GridFS — це спеціальна файлова система MongoDB для зберігання великих файлів. Вона розбиває файли на менші частини (chunk) і зберігає кожен chunk як окремий документ.

GridFS зберігає файли у двох колекціях:

- **fs.chunks** - зберігає двійкові блоки
- **fs.files** - зберігає метадані файлу

За замовчуванням розмір частини в GridFS становить 255 КБ, але ви можете задати інший розмір під час створення **bucket**. Оптимальний розмір частини може залежати від типу та розміру файлів, які ви зберігаєте, а також від ресурсів вашого обладнання та системи.

Зберігання даних: GridFS

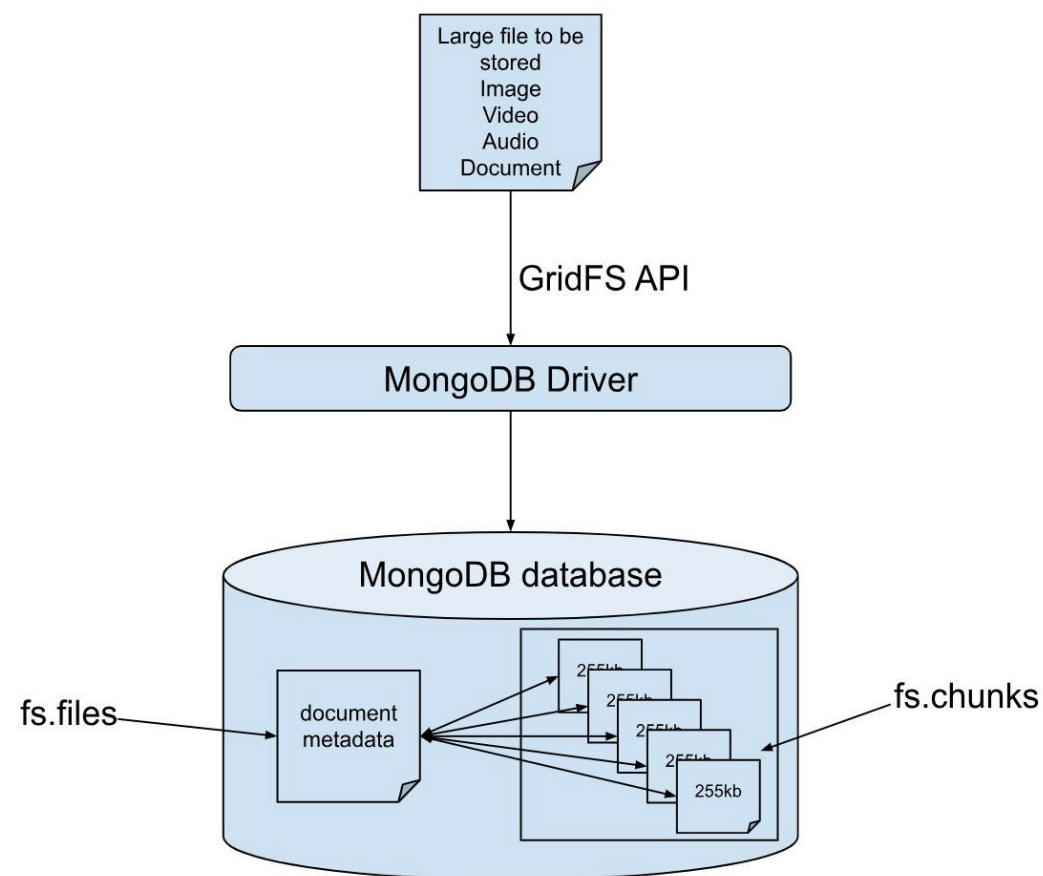
Коли ви робите запит до GridFS для отримання файлу, драйвер збирає chunk в міру необхідності.

Ви можете виконувати діапазонні запити на файли, збережені через GridFS.

Ви також можете отримувати доступ до інформації з довільних частин файлів, наприклад, щоб "перейти" до середини відео або аудіофайлу.

GridFS корисний не тільки для зберігання файлів, які перевищують 16 МБ, але й для зберігання будь-яких файлів, до яких ви хочете мати доступ без необхідності завантажувати весь файл у пам'ять.

Якщо всі ваші файли менші за обмеження розміру документа BSON у 16 МБ, спробуйте зберегти кожен файл в одному документі замість використання GridFS.



Зберігання даних: GridFS

GridFS автоматично створює індекси на колекціях `fs.files` і `fs.chunks` для забезпечення швидкого доступу до файлів і їх частин.

Індекс на колекції `fs.files`:

Індекс за полем `filename`:

```
{ filename: 1, uploadDate: 1 }
```

Цей індекс забезпечує швидкий пошук файлів за їх іменами та датою завантаження.

Індекси на колекції `fs.chunks`:

Індекс за полем `files_id` і `n`:

```
{ files_id: 1, n: 1 }
```

Цей індекс забезпечує швидкий доступ до частин, що належать певному файлу, і дозволяє швидко збирати файл з його частин.

Зберігання даних: GridFS

“images” GridFS Bucket consists of 2 regular collections.

images.files

```
1 {
2   "_id" : ObjectId("618eba4f586fadfc97b8c884"),
3   "length" : NumberInt(1610240),
4   "chunkSize" : NumberInt(261120),
5   "uploadDate" : ISODate("2021-11-12T19:02:39.726+0000"),
6   "filename" : "20211112_135041.jpg",
7   "md5" : "302010775ad0d41170db25a7e10bcda8",
8   "contentType" : "image/jpeg",
9   "metadata" : {
10     "myCustomField" : "some value",
11     "myOtherField" : {
12       "somethingElse" : 1.0
13     }
14   }
15 }
```

uploadDate field can be used to manage TTL:

```
db.getCollection("files.files").createIndex(
  {"uploadDate": 1},
  {expireAfterSeconds: 3600}
);
```

However, this will leave behind unreferenced chunks in images.chunks. To work around that, you need to set an uploadDate field in the chunks collection right after you load the file into the GridFS. In addition, you also need a TTL index on that field. Give it a slightly longer expiry period than what you specify in the .files collection - at least one extra minute as the MongoDB TTL cleanup runs about once a minute.

```
db.getCollection("files.chunks").createIndex(
  {"uploadDate": 1},
  {expireAfterSeconds: 3600+60}
);
```

“files_id” in
.chunks is foreign
key to “_id” in
.files.

images.chunks

```
1 {
2   "_id" : ObjectId("618eba4f586fad7379b8c887"),
3   "files_id" : ObjectId("618eba4f586fadfc97b8c884"),
4   "n" : NumberInt(0),
5   "data" : BinData(0, "/9j/4ZkDRXhpZgAASUkqAAgAAAAMAAAB.. 261096 more bytes")
6 }
7 {
8   "_id" : ObjectId("618eba4f586fad143fb8c888"),
9   "files_id" : ObjectId("618eba4f586fadfc97b8c884"),
10  "n" : NumberInt(1),
11  "data" : BinData(0, "zMcaTIPVsJk5GBtPP4UWLTeahhYQurQ.. 261096 more bytes")
12 }
13 {
14   "_id" : ObjectId("618eba4f586fad4bbdb8c889"),
15   "files_id" : ObjectId("618eba4f586fadfc97b8c884"),
16   "n" : NumberInt(2),
17   "data" : BinData(0, "VVJPQ8lvvh9aoCILIoHQkw+WJVz1JB6g.. 261096 more bytes")
18 }
19 {
20   "_id" : ObjectId("618eba4f586fadb851b8c88a"),
21   "files_id" : ObjectId("618eba4f586fadfc97b8c884"),
22   "n" : NumberInt(3),
23   "data" : BinData(0, "gCmKffcVG0q3XNPJ6dBjJxjJoAe0PUZB.. 261096 more bytes")
24 }
25 {
26   "_id" : ObjectId("618eba4f586fadccf1b8c88b"),
27   "files_id" : ObjectId("618eba4f586fadfc97b8c884"),
28   "n" : NumberInt(4),
29   "data" : BinData(0, "gccgKBgYwM0Yb/abIG4AZ+WgQoXtySoy.. 261096 more bytes")
30 }
31 {
32   "_id" : ObjectId("618eba4f586fad8f5fb8c88c"),
33   "files_id" : ObjectId("618eba4f586fadfc97b8c884"),
34   "n" : NumberInt(5),
35   "data" : BinData(0, "roAf/CemWYjaDyPXp3pMdep5BAPP/wCo.. 261096 more bytes")
36 }
37 {
38   "_id" : ObjectId("618eba4f586fad1de3b8c88d"),
39   "files_id" : ObjectId("618eba4f586fadfc97b8c884"),
40   "n" : NumberInt(6),
41   "data" : BinData(0, "vbV5XG0DTJG9B2IJ044w2PxrB1rPlPtZ.. 43496 more bytes")
42 }
```

Зберігання даних: URL

Якщо файл вже зберігається на зовнішньому сервері або в хмарному сховищі, ви можете зберігати лише URL у MongoDB. Це зменшує навантаження на базу даних і спрощує управління файлами.

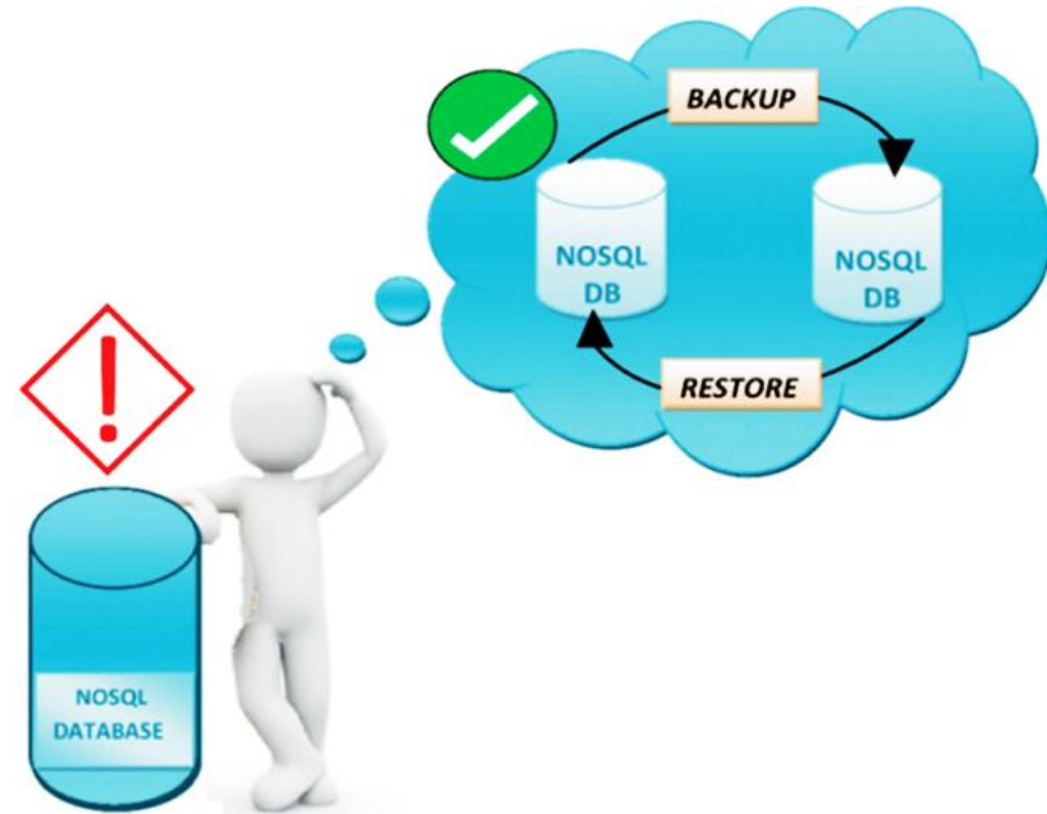
```
{  
  "_id": ObjectId("615f1b2e8f1b2c6d1f1b2c6d"),  
  "filename": "example.jpg",  
  "url": "https://example.com/path/to/example.jpg",  
  "uploadDate": ISODate("2024-10-01T00:00:00Z")  
}
```

Резервна копія та відновлення даних в MongoDB

Резервна копія та відновлення даних в MongoDB

Резервне копіювання та відновлення даних є критично важливими процесами для забезпечення безпеки та доступності даних у будь-якій системі управління базами даних (СУБД).

У контексті MongoDB, ці процеси дозволяють зберігати копії ваших даних і відновлювати їх у разі втрати, пошкодження або інших непередбачених обставин.



Резервна копія та відновлення даних в MongoDB

mongoexport та **mongoimport** — це інструменти командного рядка, які дозволяють експортувати та імпортувати дані з MongoDB у формати JSON або CSV. Людино-читабельний код.

mongodump та **mongorestore** — це інструменти командного рядка для створення резервних копій у форматі BSON та відновлення даних з цих копій.



mongoexport

```
mongoexport \
--uri="mongodb://localhost:27017" \
--username="yourUsername" \
--password="yourPassword" \
--authenticationDatabase="admin" \
--db="mydatabase" \
--collection="mycollection" \
--query='{ "field": "value" }' \
--type="json" \
--fields="field1,field2,field3" \
--sort='{ "field": 1 }' \
--limit=100 \
--skip=10 \
--out="/path/to/export.json" \
--jsonArray \
--pretty \
--authenticationMechanism="SCRAM-SHA-256" \
--ssl \
--sslCAFile="/path/to/ca.pem" \
--sslPEMKeyFile="/path/to/client.pem" \
--sslPEMKeyPassword="yourPEMKeyPassword" \
--sslCRLFile="/path/to/crl.pem" \
--sslAllowInvalidCertificates \
--sslAllowInvalidHostnames \
--sslFIPSMode
```

--uri: Вказує URL підключення до MongoDB.

--username: Вказує ім'я користувача для аутентифікації.

--password: Вказує пароль для аутентифікації.

--authenticationDatabase: Вказує базу даних для аутентифікації (зазвичай **admin**).

--db: Вказує назву бази даних.

--collection: Вказує назву колекції.

--query: Вказує фільтр для експорту даних у форматі JSON. Наприклад, { "field": "value" }.

--type: Вказує формат експорту (json або csv).

--fields: Вказує поля, які потрібно експортувати (для CSV). Наприклад, field1,field2,field3.

--sort: Вказує порядок сортування документів у форматі JSON. Наприклад, { "field": 1 }.

--limit: Вказує максимальну кількість документів для експорту.

--skip: Вказує кількість документів, які потрібно пропустити перед початком експорту.

--out: Вказує файл, у який буде збережено експортовані дані.

--jsonArray: Вказує, що дані повинні бути експортовані у вигляді масиву JSON.

--pretty: Форматує JSON з відступами для кращої читабельності.

--authenticationMechanism: Вказує механізм аутентифікації (наприклад, **SCRAM-SHA-256**).

--ssl: Включає SSL для підключення.

--sslCAFile: Вказує файл сертифіката центру сертифікації (CA).

--sslPEMKeyFile: Вказує файл сертифіката клієнта (PEM).

--sslPEMKeyPassword: Вказує пароль для файлу сертифіката клієнта (PEM).

--sslCRLFile: Вказує файл списку відкликаних сертифікатів (CRL).

--sslAllowInvalidCertificates: Дозволяє використання недійсних сертифікатів.

--sslAllowInvalidHostnames: Дозволяє використання недійсних імен хостів.

--sslFIPSMode: Включає режим FIPS для SSL.

mongoimport

```
mongoimport \
--uri="mongodb://localhost:27017" \
--username="yourUsername" \
--password="yourPassword" \
--authenticationDatabase="admin" \
--db="mydatabase" \
--collection="mycollection" \
--file="/path/to/import.json" \
--type="json" \
--fields="field1,field2,field3" \
--headerline \
--jsonArray \
--drop \
--upsert \
--upsertFields="field1,field2" \
--authenticationMechanism="SCRAM-SHA-256" \
--ssl \
--sslCAFile="/path/to/ca.pem" \
--sslPEMKeyFile="/path/to/client.pem" \
--sslPEMKeyPassword="yourPEMKeyPassword" \
--sslCRLFile="/path/to/crl.pem" \
--sslAllowInvalidCertificates \
--sslAllowInvalidHostnames \
--sslFIPSMode
```

--uri: Вказує URL підключення до MongoDB.

--username: Вказує ім'я користувача для аутентифікації.

--password: Вказує пароль для аутентифікації.

--authenticationDatabase: Вказує базу даних для аутентифікації (зазвичай **admin**).

--db: Вказує назву бази даних.

--collection: Вказує назву колекції.

--file: Вказує файл, з якого будуть імпортовані дані.

--type: Вказує формат імпорту (json або csv).

--fields: Вказує поля, які потрібно імпортувати (для CSV).

--headerline: Вказує, що перший рядок CSV файлу містить назви полів (для CSV).

--jsonArray: Вказує, що дані у файлі JSON представлені у вигляді масиву.

--drop: Видаляє існуючу колекцію перед імпортом нових даних.

--upsert: Виконує операцію upsert (оновлення або вставка) для кожного документа.

--upsertFields: Вказує поля для операції upsert.

--authenticationMechanism: Вказує механізм аутентифікації (наприклад, **SCRAM-SHA-256**).

--ssl: Включає SSL для підключення.

--sslCAFile: Вказує файл сертифіката центру сертифікації (CA).

--sslPEMKeyFile: Вказує файл сертифіката клієнта (PEM).

--sslPEMKeyPassword: Вказує пароль для файлу сертифіката клієнта (PEM).

--sslCRLFile: Вказує файл списку відкликаних сертифікатів (CRL).

--sslAllowInvalidCertificates: Дозволяє використання недійсних сертифікатів.

--sslAllowInvalidHostnames: Дозволяє використання недійсних імен хостів.

--sslFIPSMode: Включає режим FIPS для SSL.

mongoexport та mongoimport

Експорт даних з MongoDB у форматі JSON

```
mongoexport \  
  --uri="mongodb://localhost:27017" \  
  --db="mydatabase" \  
  --collection="mycollection" \  
  --out="export.json" \  
  --jsonArray
```

Приклад даних у файлі import.json

```
[  
  { "name": "Alice", "age": 30, "city": "New York" },  
  { "name": "Bob", "age": 25, "city": "San Francisco" },  
  { "name": "Charlie", "age": 35, "city": "Los Angeles" }  
]
```

Імпорт даних у MongoDB з формату JSON

```
mongoimport \  
  --uri="mongodb://localhost:27017" \  
  --db="mydatabase" \  
  --collection="mycollection" \  
  --file="import.json" \  
  --jsonArray
```

Ці прості приклади показують, як використовувати **mongoexport** та **mongoimport** для експорту та імпорту даних у форматі JSON у MongoDB.

Ви можете налаштувати команди відповідно до ваших потреб, використовуючи доступні параметри.

mongoexport та mongoimport

Експорт даних з MongoDB у форматі CSV

```
mongoexport \  
--uri="mongodb://localhost:27017" \  
--db="mydatabase" \  
--collection="mycollection" \  
--type="csv" \  
--fields="name,age,city" \  
--out="export.csv"
```

Приклад даних у файлі import.csv

```
name,age,city  
Alice,30,New York  
Bob,25,San Francisco  
Charlie,35,Los Angeles
```

Імпорт даних у MongoDB з формату CSV

```
mongoimport \  
--uri="mongodb://localhost:27017" \  
--db="mydatabase" \  
--collection="mycollection" \  
--type="csv" \  
--file="import.csv" \  
--headerline
```

Ці прості приклади показують, як використовувати **mongoexport** та **mongoimport** для експорту та імпорту даних у форматі CSV у MongoDB.

Ви можете налаштувати команди відповідно до ваших потреб, використовуючи доступні параметри.

mongodump

```
mongodump \
--uri="mongodb://localhost:27017" \
--username="yourUsername" \
--password="yourPassword" \
--authenticationDatabase="admin" \
--db="mydatabase" \
--collection="mycollection" \
--query='{ "field": "value" }' \
--out="/path/to/backup" \
--gzip \
--archive="/path/to/backup/archive.gz" \
--authenticationMechanism="SCRAM-SHA-256" \
--ssl \
--sslCAFile="/path/to/ca.pem" \
--sslPEMKeyFile="/path/to/client.pem" \
--sslPEMKeyPassword="yourPEMKeyPassword" \
--sslCRLFile="/path/to/crl.pem" \
--sslAllowInvalidCertificates \
--sslAllowInvalidHostnames \
--sslFIPSMode
```

--uri: Вказує URL підключення до MongoDB.

--username: Вказує ім'я користувача для аутентифікації.

--password: Вказує пароль для аутентифікації.

--authenticationDatabase: Вказує базу даних для аутентифікації (зазвичай **admin**).

--db: Вказує назву бази даних.

--collection: Вказує назву колекції.

--query: Вказує фільтр для резервного копіювання даних у форматі JSON. Наприклад, {
"field": "value"}.

--out: Вказує директорію, куди буде збережено дампу.

--gzip: Стискає дампу за допомогою gzip.

--archive: Вказує файл архіву для збереження дампу.

--authenticationMechanism: Вказує механізм аутентифікації (наприклад, **SCRAM-SHA-256**).

--ssl: Включає SSL для підключення.

--sslCAFile: Вказує файл сертифіката центру сертифікації (CA).

--sslPEMKeyFile: Вказує файл сертифіката клієнта (PEM).

--sslPEMKeyPassword: Вказує пароль для файлу сертифіката клієнта (PEM).

--sslCRLFile: Вказує файл списку відкликаних сертифікатів (CRL).

--sslAllowInvalidCertificates: Дозволяє використання недійсних сертифікатів.

--sslAllowInvalidHostnames: Дозволяє використання недійсних імен хостів.

--sslFIPSMode: Включає режим FIPS для SSL.

mongorestore

```
mongorestore \  
--uri="mongodb://localhost:27017" \  
--username="yourUsername" \  
--password="yourPassword" \  
--authenticationDatabase="admin" \  
--db="mydatabase" \  
--collection="mycollection" \  
--dir="/path/to/backup" \  
--gzip \  
--archive="/path/to/backup/archive.gz" \  
--drop \  
--nsInclude="mydatabase.mycollection" \  
--nsExclude="mydatabase.othercollection" \  
--preserveUUID \  
--stopOnError \  
--noIndexRestore \  
--maintainInsertionOrder \  
--numInsertionWorkersPerCollection=4 \  
--numParallelCollections=4 \  
--authenticationMechanism="SCRAM-SHA-256" \  
--ssl \  
--sslCAFile="/path/to/ca.pem" \  
--sslPEMKeyFile="/path/to/client.pem" \  
--sslPEMKeyPassword="yourPEMKeyPassword" \  
--sslCRLFile="/path/to/crl.pem" \  
--sslAllowInvalidCertificates \  
--sslAllowInvalidHostnames \  
--sslFIPSMode
```

--uri: Вказує URL підключення до MongoDB.

--username: Вказує ім'я користувача для аутентифікації.

--password: Вказує пароль для аутентифікації.

--authenticationDatabase: Вказує базу даних для аутентифікації (зазвичай **admin**).

--db: Вказує назву бази даних.

--collection: Вказує назву колекції.

--dir: Вказує директорію, з якої будуть відновлені дані.

--gzip: Вказує, що дані стиснуті за допомогою gzip.

--archive: Вказує файл архіву для відновлення даних.

--drop: Видаляє існуючі дані перед відновленням.

--nsInclude: Включає тільки вказані простори імен (namespace) для відновлення.

--nsExclude: Виключає вказані простори імен (namespace) з відновлення.

--preserveUUID: Зберігає UUID колекцій під час відновлення.

--stopOnError: Зупиняє відновлення при виникненні помилки.

--noIndexRestore: Не відновлює індекси.

--maintainInsertionOrder: Зберігає порядок вставки документів.

--numInsertionWorkersPerCollection: Вказує кількість робітників для вставки документів у кожен колекцію.

--numParallelCollections: Вказує кількість колекцій, які будуть відновлюватися паралельно.

--authenticationMechanism: Вказує механізм аутентифікації (наприклад, **SCRAM-SHA-256**).

--ssl: Включає SSL для підключення.

--sslCAFile: Вказує файл сертифіката центру сертифікації (CA).

--sslPEMKeyFile: Вказує файл сертифіката клієнта (PEM).

--sslPEMKeyPassword: Вказує пароль для файлу сертифіката клієнта (PEM).

--sslCRLFile: Вказує файл списку відкликаних сертифікатів (CRL).

--sslAllowInvalidCertificates: Дозволяє використання недійсних сертифікатів.

--sslAllowInvalidHostnames: Дозволяє використання недійсних імен хостів.

--sslFIPSMode: Включає режим FIPS для SSL.

mongodump та mongorestore

Створення резервної копії всієї бази даних

```
mongodump --uri="mongodb://localhost:27017" --db="mydatabase" --out="backup"
```

Відновлення всієї бази даних з резервної копії

```
mongorestore --uri="mongodb://localhost:27017" --db="mydatabase" --dir="backup/mydatabase"
```

Створення резервної копії окремої колекції

```
mongodump --uri="mongodb://localhost:27017" --db="mydatabase" --collection="mycollection" --out="backup"
```

Відновлення окремої колекції з резервної копії

```
mongorestore --uri="mongodb://localhost:27017" --db="mydatabase" --collection="mycollection"  
--dir="backup/mydatabase/mycollection.bson"
```

Приклад структури дампу

mongodump

--uri="mongodb://localhost:27017"

--db="mydatabase" --out="backup"

Після виконання цієї команди структура директорії backup буде виглядати приблизно так:

```
backup/
├── mydatabase/
│   ├── users.bson
│   ├── users.metadata.json
│   ├── orders.bson
│   └── orders.metadata.json
```

- **backup/**: Директорія, в яку збережено дамп.
- **mydatabase/**: Піддиректорія, яка відповідає назві бази даних.
 - **users.bson**: Файл, що містить дані колекції **users** у форматі BSON.
 - **users.metadata.json**: Файл, що містить метадані колекції **users**, такі як індекси.
 - **orders.bson**: Файл, що містить дані колекції **orders** у форматі BSON.
 - **orders.metadata.json**: Файл, що містить метадані колекції **orders**, такі як індекси.

Приклад даних у форматі **JSON**, які можуть бути збережені у **users.bson**

```
[
  { "_id": { "$oid": "60c72b2f9b1d8b3f4c8e4d1a" }, "name": "Alice", "email": "alice@example.com", "age": 30 },
  { "_id": { "$oid": "60c72b2f9b1d8b3f4c8e4d1b" }, "name": "Bob", "email": "bob@example.com", "age": 25 },
  { "_id": { "$oid": "60c72b2f9b1d8b3f4c8e4d1c" }, "name": "Charlie", "email": "charlie@example.com", "age": 35 }
]
```

Приклад вмісту **users.metadata.json**

```
{
  "options": {
    "capped": false,
    "size": 5242880,
    "max": 5000
  },
  "indexes": [
    {
      "v": 2,
      "key": { "_id": 1 },
      "name": "_id_",
      "ns": "mydatabase.users"
    },
    {
      "v": 2,
      "key": { "email": 1 },
      "name": "email_1",
      "ns": "mydatabase.users",
      "unique": true
    },
    {
      "v": 2,
      "key": { "age": 1 },
      "name": "age_1",
      "ns": "mydatabase.users"
    }
  ],
  "uuid": "c1a2b3c4-d5e6-7f8a-9b0c-d1e2f3a4b5c6"
}
```

Перенесення даних з SQL до MongoDB

Перенесення даних з SQL БД до MongoDB може бути виконане за допомогою різних методів, включаючи інструменти командного рядка, скрипти на Python, PHP, Ruby, Java, C#, ETL інструменти та інструменти міграції даних.

Вибір методу залежить від ваших конкретних вимог та обсягу даних, які потрібно перенести.

Використання інструменту mongoimport та експорту CSV з MySQL

Експорт даних з MySQL у форматі CSV:

Використовуйте команду SELECT ... INTO OUTFILE для експорту даних з MySQL у файл CSV.

```
SELECT * FROM mytable INTO OUTFILE '/path/to/export.csv'  
FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n';
```

Імпорт даних у MongoDB за допомогою mongoimport:

Використовуйте команду mongoimport для імпорту даних з файлу CSV у MongoDB.

```
mongoimport --uri="mongodb://localhost:27017" --db="mydatabase"  
--collection="mycollection" --type="csv" --file="/path/to/export.csv" --headerline
```

Безпека в MongoDB

Безпека MongoDB

Безпека MongoDB є критично важливою для захисту даних від несанкціонованого доступу, втрати або пошкодження. MongoDB надає різні механізми безпеки, які допомагають забезпечити конфіденційність, цілісність та доступність даних.

Основні аспекти безпеки MongoDB:

Аутентифікація:

Забезпечує перевірку особи користувача або програми, яка намагається отримати доступ до БД.

Авторизація:

Визначає, які дії користувач або програма можуть виконувати після успішної аутентифікації.

Шифрування:

Забезпечує захист даних як під час зберігання, так і під час передачі.

Аудит:

Дозволяє відстежувати дії користувачів та події в системі для забезпечення відповідності політикам безпеки та виявлення потенційних загроз.

Мережеві налаштування:

Допомагають обмежити доступ до MongoDB лише авторизованим користувачам та системам.

Конфігурація безпеки:

Налаштування параметрів безпеки у файлі `mongod.conf`, включаючи параметри для аутентифікації, авторизації, шифрування та мережевих налаштувань.

Кращі практики:

Використання сильних паролів, регулярне оновлення MongoDB, моніторинг та аудит, резервне копіювання, обмеження доступу.



Безпека MongoDB

Аутентифікація

Аутентифікація забезпечує перевірку особи користувача або програми, яка намагається отримати доступ до бази даних.

SCRAM (Salted Challenge Response Authentication Mechanism):

Основний механізм аутентифікації в MongoDB. Підтримує SCRAM-SHA-1 та SCRAM-SHA-256.

x.509 Certificates: Використовується для аутентифікації за допомогою сертифікатів x.509.

LDAP (Lightweight Directory Access Protocol): Дозволяє інтеграцію з існуючими LDAP-серверами для централізованої аутентифікації.

Kerberos: Підтримує аутентифікацію за допомогою протоколу Kerberos.

Безпека MongoDB

Авторизація

Авторизація визначає, які дії користувач або програма можуть виконувати після успішної аутентифікації.

Ролі користувачів: MongoDB використовує ролі для визначення дозволів. Існують вбудовані ролі, такі як *read*, *readWrite*, *dbAdmin*, а також можливість створення користувацьких ролей.

Гранулярні дозволи: Дозволяють налаштовувати доступ до окремих колекцій, баз даних або навіть окремих операцій.

Безпека MongoDB

Шифрування

Шифрування забезпечує захист даних як під час зберігання, так і під час передачі.

Шифрування даних у стані спокою (Encryption at Rest):

Використовується для шифрування даних, що зберігаються на диску. MongoDB Enterprise пропонує вбудоване шифрування даних у стані спокою.

Шифрування даних під час передачі (Encryption in Transit):

Використовується для шифрування даних, що передаються між клієнтами та серверами MongoDB. Підтримується SSL/TLS.

Безпека MongoDB

Аудит

Аудит дозволяє відстежувати дії користувачів та події в системі для забезпечення відповідності політикам безпеки та виявлення потенційних загроз.

Аудит подій: MongoDB Enterprise підтримує аудит подій, таких як аутентифікація, авторизація, операції з даними та зміни конфігурації.

Безпека MongoDB

Мережеві налаштування

Мережеві налаштування допомагають обмежити доступ до MongoDB лише авторизованим користувачам та системам.

Брандмауери та списки контролю доступу (ACL):

Використовуються для обмеження доступу до MongoDB з певних IP-адрес або діапазонів IP-адрес.

Відключення відкритого доступу: Рекомендується відключити відкритий доступ до MongoDB з Інтернету та дозволити доступ лише з внутрішньої мережі або через VPN.

Безпека MongoDB

Конфігурація безпеки

Конфігурація безпеки MongoDB включає налаштування параметрів безпеки у файлі конфігурації MongoDB (mongod.conf).

Приклад конфігурації mongod.conf:

security:

authorization: enabled

keyFile: /path/to/keyfile

net:

ssl:

mode: requireSSL

PEMKeyFile: /path/to/mongodb.pem

CAFile: /path/to/ca.pem

bindIp: 127.0.0.1

Безпека MongoDB

Кращі практики безпеки

Використовуйте сильні паролі: Забезпечте використання сильних паролів для всіх користувачів MongoDB.

Регулярно оновлюйте MongoDB: Встановлюйте останні оновлення та патчі безпеки.

Моніторинг та аудит: Регулярно моніторте та аудитуйте дії користувачів та події в системі.

Резервне копіювання: Регулярно створюйте резервні копії даних та перевіряйте їх відновлюваність.

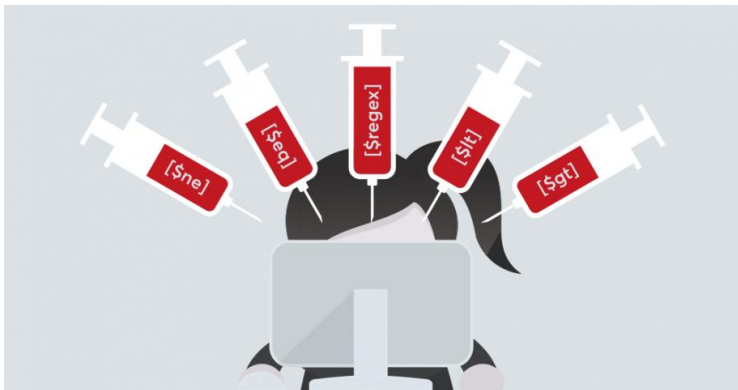
Обмежте доступ: Використовуйте принцип найменших привілеїв для користувачів та ролей.

Ін'єкційна атака

Ін'єкція (ін'єкційна атака) є одним з найпоширеніших типів атак на веб-додатки, під час якого зловмисник вводить шкідливий код або команди в запити до бази даних або інших систем, щоб отримати несанкціонований доступ до даних або виконати небажані дії.

У контексті MongoDB, ін'єкційні атаки можуть бути спрямовані на запити до бази даних, що використовують MongoDB Query Language (MQL).

Як працює ін'єкція в MongoDB



Ін'єкційні атаки в MongoDB зазвичай відбуваються, коли додаток приймає введення від користувача і безпосередньо використовує його у запитах до бази даних без належної валідації, обробки та фільтрації введених даних. Це може дозволити зловмиснику вставити шкідливий код у запит, що призведе до небажаних результатів.

Приклад ін'єкційної атаки

Припустимо, у вас є веб-додаток, який приймає ім'я користувача через форму і використовує його для пошуку користувача в базі даних MongoDB:

```
const username = req.body.username;  
const user = await db.collection('users').findOne({ username: username });
```

Якщо зловмисник введе наступне значення для **username**:

```
{ "$ne": null }
```

Це призведе до того, що запит знайде першого користувача в колекції, оскільки **\$ne** (не дорівнює) є оператором MongoDB, який поверне всі документи, де **username** не дорівнює **null**.

Захист від ін'єкційних атак

Валідація та очищення введення:

Завжди перевіряйте, обробляйте та фільтруйте введені дані від користувачів перед використанням його у запитах до бази даних. Використовуйте бібліотеки для валідації введення, такі як `validator` у `Node.js`.

Використання параметризованих запитів:

Використовуйте параметризовані запити або підготовлені вирази, які автоматично обробляють введення від користувачів як дані, а не як частину запиту.

Використання бібліотек ORM/ODM:

Використовуйте бібліотеки ORM (Object-Relational Mapping) або ODM (Object-Document Mapping), такі як `Mongoose` для `Node.js`, які допомагають автоматично обробляти введення від користувачів і зменшують ризик ін'єкційних атак.

Обмеження прав доступу:

Обмежте права доступу до бази даних для користувачів та додатків. Використовуйте принцип найменших привілеїв, щоб зменшити можливість виконання шкідливих дій.

Моніторинг та аудит:

Регулярно моніторте та аудитуйте запити до бази даних для виявлення підозрілої активності. Використовуйте інструменти для моніторингу бази даних, такі як `MongoDB Atlas`.

Приклад захищеного коду

```
const validator = require('validator');

const username = req.body.username;

// Валідація введення
if (!validator.isAlphanumeric(username)) {
  return res.status(400).send('Invalid username');
}

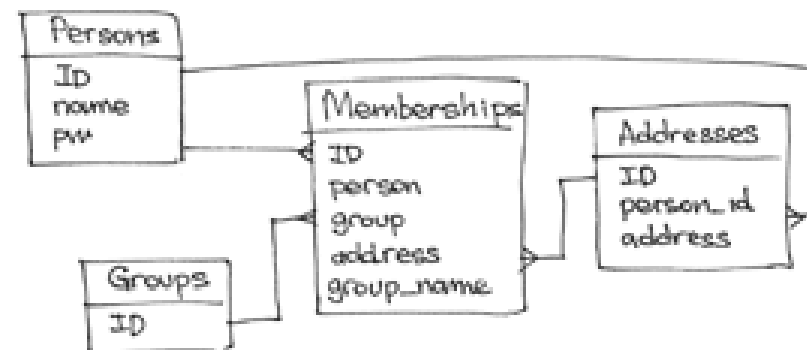
const user = await
db.collection('users').findOne({ username:
username });
```

Моделювання даних в MongoDB

Моделювання даних в MongoDB

Модель даних — це візуальне представлення того, як ви будете зберігати дані, а також те, як дані пов'язані одне з одним.

Мета створення моделі даних полягає в тому, щоб ідентифікувати всі компоненти даних системи та те, як вони пов'язані між собою.



Моделювання даних у MongoDB відрізняється від реляційних баз даних через гнучкість і можливість зберігання даних у вигляді документів. Це дозволяє створювати більш природні та ефективні моделі даних для багатьох застосувань.

Основні принципи моделювання даних

Вбудовані документи (Embedded Documents), денормалізована

Використовуються для зберігання пов'язаних даних у одному документі, що дозволяє уникнути об'єднань (joins) і покращити продуктивність.

Вбудовування передбачає зберігання пов'язаних даних у вигляді вкладених документів.

Це підходить для сценаріїв, де дані мають тісний зв'язок і часто використовуються разом.

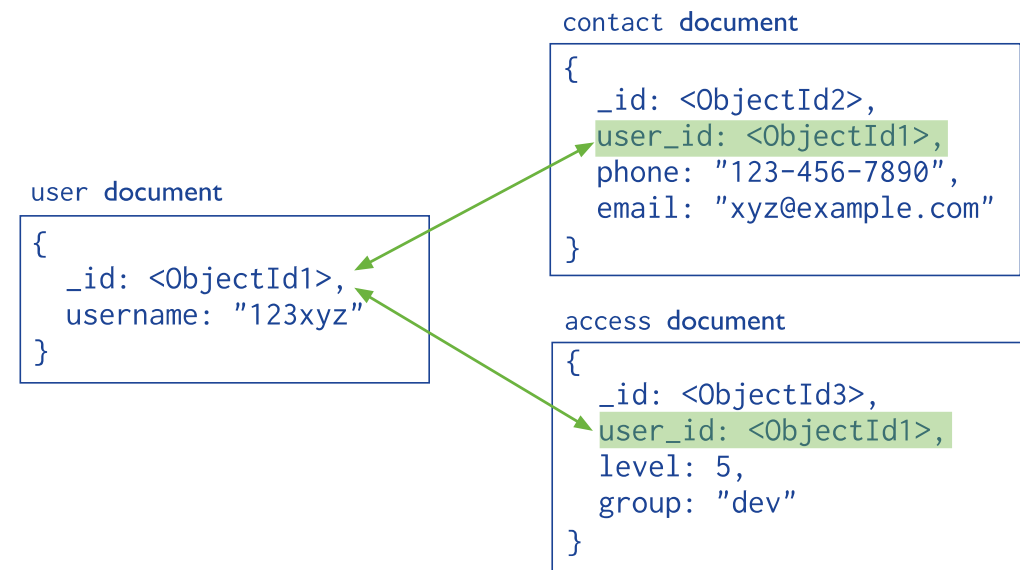


Посилання (References), нормалізована

Використовуються для зберігання відносин між документами у різних колекціях, що дозволяє зберігати дані окремо і зменшити дублювання.

Посилання передбачає зберігання ідентифікаторів пов'язаних документів.

Це підходить для сценаріїв, де дані мають слабкий зв'язок або часто змінюються.

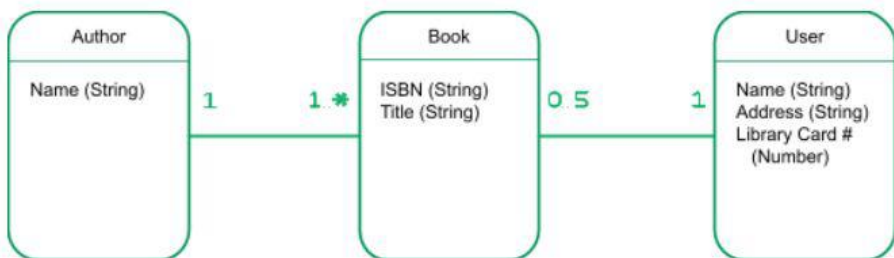


Типи моделей даних



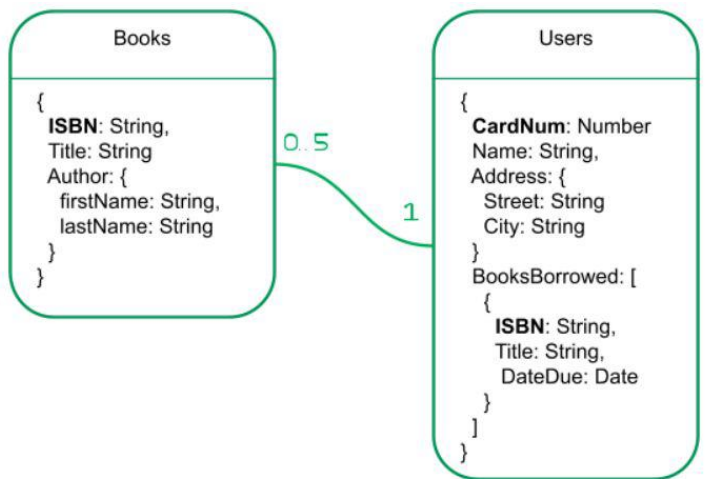
Концептуальна модель даних:

Пояснює, що має містити система щодо даних і як вони пов'язані. Представляє бізнес-логіку програми та використовується як основа для інших моделей.



Логічна модель даних:

Описує, як дані будуть структуровані. Встановлює зв'язки між сутностями на високому рівні та перераховує атрибути для сутностей.



Фізична модель даних:

Представляє, як дані будуть зберігатися в MongoDB. Вирішує, чи вбудовувати чи зв'язувати дані, встановлює типи даних для полів і надає схему бази даних.

Приклад моделювання даних у MongoDB для блогу

Концептуальна модель даних

Концептуальна модель даних визначає основні сутності та їхні взаємозв'язки.

У нашому прикладі блогу ми маємо такі сутності:

- Користувачі (Users): Зберігає інформацію про користувачів блогу.
- Пости (Posts): Зберігає інформацію про публікації.
- Коментарі (Comments): Зберігає інформацію про коментарі до публікацій.

Взаємозв'язки:

- Користувачі можуть створювати пости.
- Користувачі можуть залишати коментарі до постів.

Логічна модель даних

Логічна модель даних описує структуру даних і зв'язки між сутностями на високому рівні.

Користувачі (Users):

Ім'я (name)
Електронна пошта (email)
Пароль (password)

Пости (Posts):

Заголовок (title)
Вміст (content)
Автор (author) - посилання на користувача
Коментарі (comments) - вбудовані документи

Коментарі (Comments):

Автор (author)
Вміст (content)
Дата (date)

Фізична модель даних

Фізична модель даних визначає, як дані будуть зберігатися в MongoDB. У нашому прикладі ми використовуємо вбудовані документи для коментарів і посилання для авторів постів.

Колекція users

```
{
  "_id": ObjectId("60c72b2f9b1d8b3f4c8e4d1a"),
  "name": "Alice",
  "email": "alice@example.com",
  "password": "hashed_password"
}
```

Колекція posts

```
{
  "_id": ObjectId("60c72b2f9b1d8b3f4c8e4d1c"),
  "title": "My First Post",
  "content": "This is the content of my first post.",
  "author": ObjectId("60c72b2f9b1d8b3f4c8e4d1a"),
  "comments": [
    {
      "author": "Bob",
      "content": "Great post!",
      "date": ISODate("2021-06-13T12:00:00Z")
    }
  ]
}
```

Моделювання даних у MongoDB

Основний принцип, який слід пам'ятати при моделюванні даних, полягає в тому, що дані, які часто запитуються разом, повинні зберігатися якомога ближче один до одного. Це означає, що вони повинні бути в одному документі або колекції. Хоча витягування даних з іншої колекції можливе, це не найефективніший спосіб використання MongoDB.

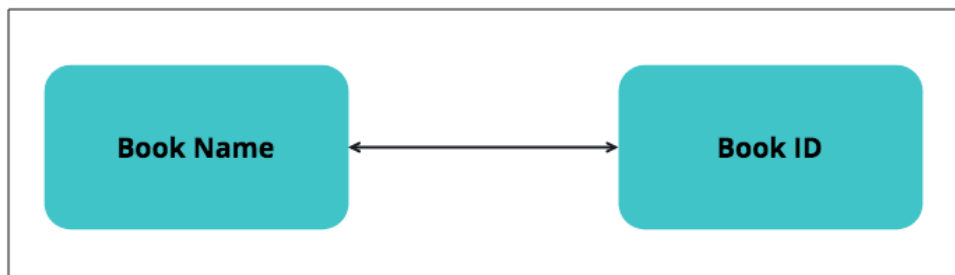
Обмеження та продуктивність

Обмеження на розмір документа: 16 мегабайт.

Обмеження на обробку в конвеєрі агрегації.

Продуктивність: Операції з великими документами менш ефективні, ніж з маленькими документами. Тому ви не завжди можете зберігати всі дані разом.

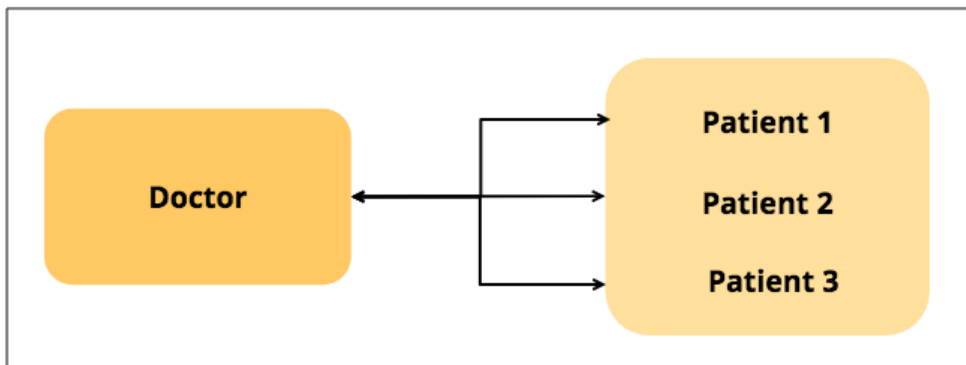
Загальні рекомендації



Відношення один-до-одного:

Зберігайте разом: Якщо існує відношення один-до-одного між документом і іншими даними, наприклад, між документом про людину і датою народження, зберігайте їх разом.

Винятки: Якщо розмір занадто великий або якщо ви дуже рідко використовуєте цю інформацію разом у запитах.

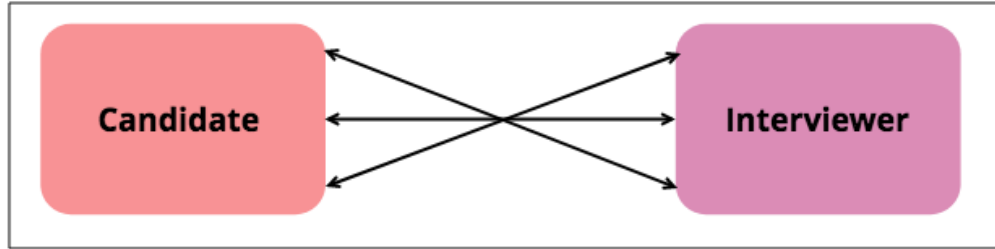


Відношення один-до-багатьох:

Малий обсяг: Якщо це відношення один-до-кількох, як у випадку з авторами і книгами, ви можете зберігати обидва в одній колекції, використовуючи вкладеність або масив.

Великий обсяг: Якщо це більше ніж кілька, найкраще зберігати їх у окремих колекціях і зв'язувати їх за допомогою ідентифікаторів.

Загальні рекомендації



Відношення багато-до-багатьох:

Уникайте вбудовування: Уникайте вбудовування цієї інформації. Замість цього зберігайте ідентифікатори в одній або обох колекціях.

Додаткові поради

Масиви: Тримайте масиви розумно маленькими, тобто менше 100 елементів.

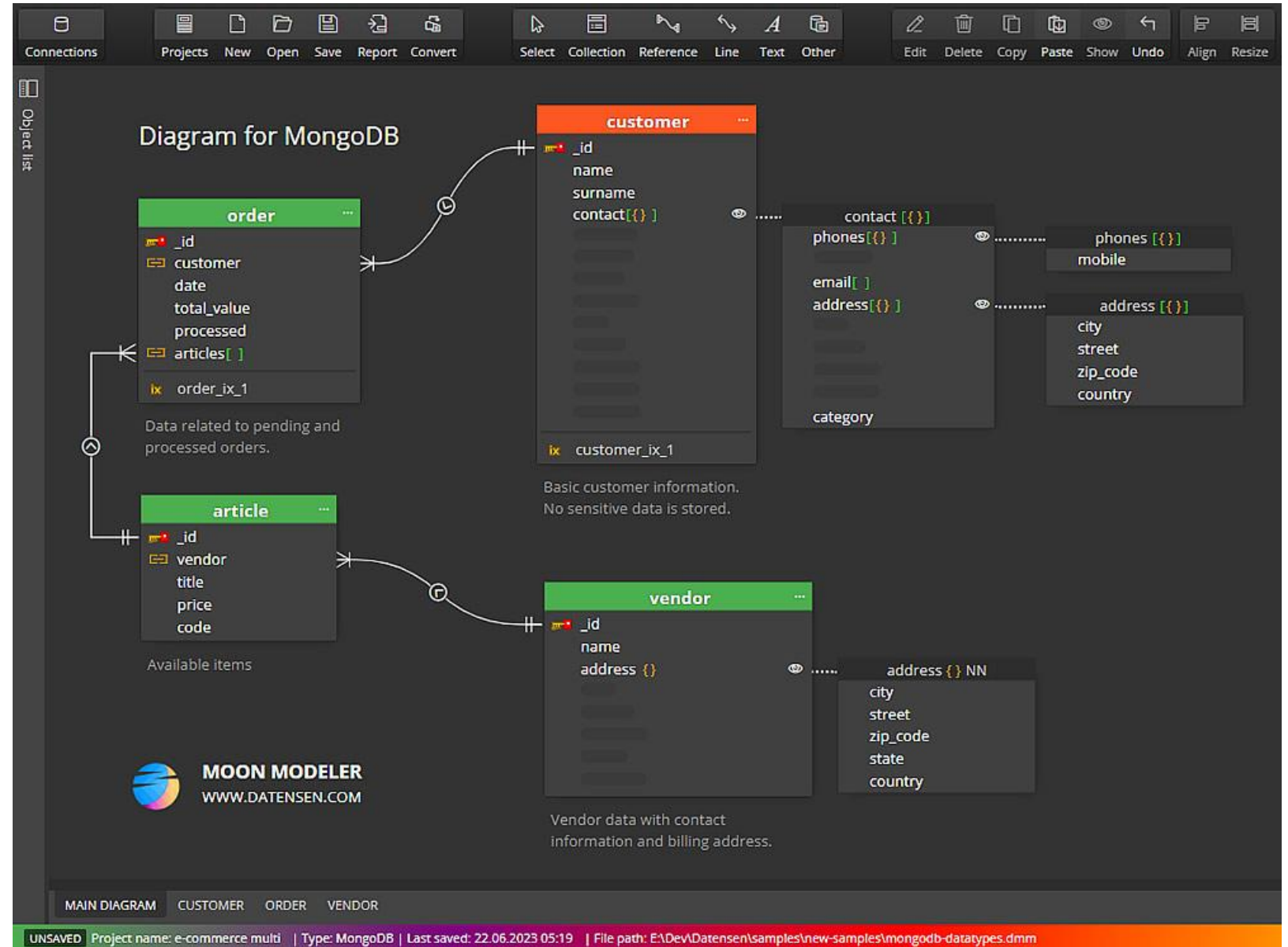
Структура документів: Документи не повинні бути великими і плоскими. Якщо у вас є 100 пар ключ-значення і мало вкладеності, це не дуже добре для продуктивності.

Рекомендації під час проектування схеми MongoDB

- **Проектуйте схему відповідно до вимог користувача.**
- **Об'єднання даних.** Якщо об'єкти часто використовуються разом, зберігайте їх в одному документі. Якщо ні, розділіть їх, але переконайтеся, що з'єднання не потрібне.
- **Дублювання даних.** Дублюйте дані (помірно), оскільки простір на диску дешевше, ніж час обчислення.
- **Операції з'єднання.** Виконуйте операції з'єднання під час запису, а не під час читання.
- **Оптимізація схеми.** Оптимізуйте схему для найчастіших випадків використання.
- **Агрегація.** Виконуйте складні операції агрегації у схемі.

Дизайн схеми MongoDB – програми

- Luna Modeler
- Moon Modeler
- Hackolade Studio
- Open ModelSphere
- Creately



ДЯКУЮ ЗА УВАГУ!